

DRIFTAUDIO: MARGINAL DRIFTING FOR DISTRIBUTIONAL POST-TRAINING OF ONE-STEP TEXT-TO-AUDIO GENERATORS

Xingyu Chen, Fei Ma, Sipei Zhao

Centre for Audio, Acoustics and Vibration, Faculty of Engineering and IT
University of Technology Sydney, Ultimo, NSW 2007, Australia

ABSTRACT

Recent one-step text-to-audio (TTA) models substantially reduce inference cost, yet their generated distributions can still be improved through post-training. We propose **DriftAudio**, a distributional post-training method that adapts Drifting to pretrained one-step TTA generators. Applying Drifting condition-wise is challenging under free-form text conditioning, where only one or a few real samples are typically available for a particular condition. DriftAudio instead performs Drifting on the marginal audio distribution while retaining text-conditioned generation. The drifting field is estimated in a frozen audio feature space using resampled real samples, a rolling bank of generated samples, and the current batch of generated samples. The resulting Drifting field provides a detached training target for updating only the generator, keeping the original one-step inference procedure. On AudioCaps, starting from MeanAudio, DriftAudio reduces FAD and FD by 33.9% and 17.6%, respectively, while also improving KL and CLAP. Starting from FdAudio, it further reduces FAD, FD, and KL, with trade-offs in IS and CLAP. These results demonstrate the effectiveness of marginal distributional post-training for one-step TTA generation.

Index Terms— text-to-audio generation, one-step generation, distributional post-training, drifting

1. INTRODUCTION

Text-to-audio (TTA) generation aims to synthesize realistic audio aligned with free-form natural-language descriptions [1]. Diffusion- and flow-based models have substantially improved generation quality [2, 3, 4, 5], but high-quality inference commonly relies on iterative sampling with repeated neural network evaluations (NFEs). This motivates low-NFE TTA generation, with strict one-step generation requiring only a single NFE.

Recent low-NFE TTA methods reduce sampling cost by compressing iterative sampling into a small number of NFEs. Consistency-based methods learn consistent mappings across states along diffusion trajectories, enabling one- or few-step synthesis [6, 7, 8]. Trajectory rectification improves few-step generation by simplifying or straightening the sampling path [9, 10]. More recently, MeanAudio [11] adopts the MeanFlow objective [12] to learn average flow dynamics over finite time intervals, enabling strict 1-NFE generation. Despite these advances, generators can still benefit from further quality refinement, motivating post-training refinement.

Distributional post-training provides a complementary route. Rather than introducing a new inference-time sampler, it directly refines the distribution induced by a pretrained generator. FdAudio [13], for example, post-trains MeanAudio [11] by matching real and generated audio distributions across multiple frozen au-

dio feature spaces using Fréchet objectives [14] based on first- and second-order moments.

Drifting [15] provides a sample-based alternative to moment-based distribution matching by constructing a distribution-dependent field from real and generated samples through kernel-weighted interactions. Drifting has recently been explored for one-step speech enhancement [16], but its use for TTA remains unexplored. A direct condition-wise application to TTA would require estimating a separate conditional data distribution for each text condition. This is challenging because each free-form text condition is typically associated with only one or a few real audio samples. Our key observation is that, after conditional generation has been learned, distributional refinement can be performed on the marginal output distribution without estimating a separate data distribution for each text condition.

Building on this observation, we propose **DriftAudio**, a distributional post-training method for pretrained one-step TTA generators. Generation remains text-conditioned, while distribution matching is performed marginally in a feature space. Real feature samples are resampled at each step from the full training set, while the generator-induced distribution is represented by a first-in, first-out (FIFO) bank initialized with pre-generated samples and refreshed using a small batch of current generated samples. The drifting field is then estimated between the current generated samples and the real and generated reference samples, and converted into a detached target to update only the generator while preserving one-step inference.

Our contributions are threefold: (i) We formulate Drifting as a marginal distributional post-training objective for one-step TTA generators, avoiding the need to estimate a separate real-audio distribution for each text condition; (ii) we develop a practical estimator using resampled real features and a FIFO bank of generated features; (iii) we demonstrate on AudioCaps that DriftAudio improves FAD, FD, and KL from both starting checkpoints, while retaining competitive text-audio alignment. To the best of our knowledge, this is the first application of Drifting to TTA generation. Audio demos are available on the project page.¹ Code and model weights will be released upon acceptance.

2. PRELIMINARIES

2.1. Pretrained One-Step TTA Generator

We use the MeanAudio architecture [11], which enables one-step TTA generation using the MeanFlow objective [12]. Unlike standard flow matching [17], which models instantaneous velocity, Mean-

¹<https://xingyuaudio.github.io/driftdaudio-demo/>

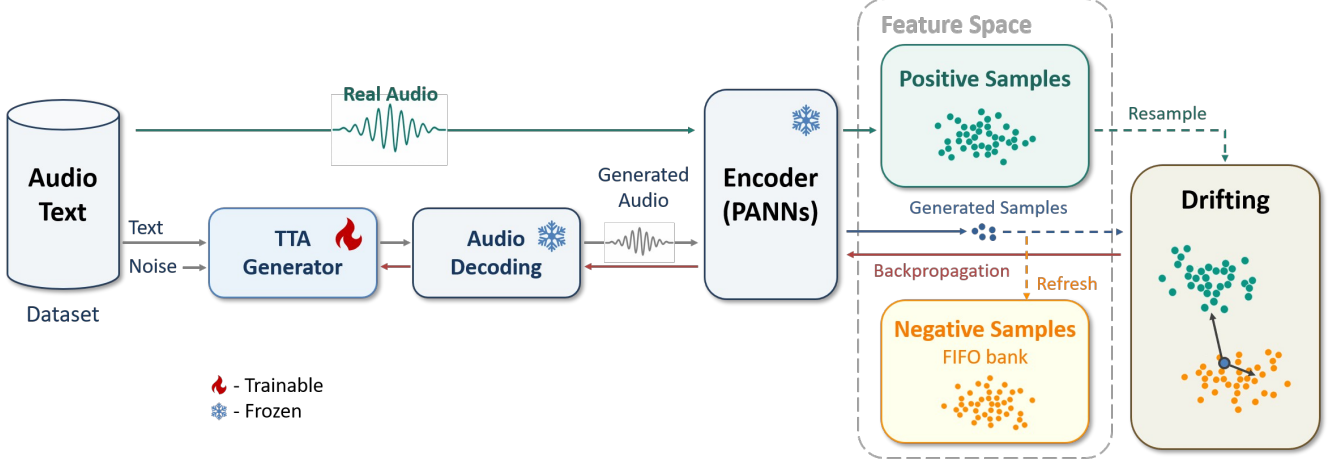


Fig. 1. Overview of DriftAudio. The one-step generator remains text-conditioned, while Drifting is performed on the marginal audio distribution in feature space. At each optimization step, real features are resampled from the training set, while generated reference features are maintained in a FIFO bank initialized with pre-generated samples and updated using the current generated samples.

Flow models the average velocity over a finite interval $[r, t]$:

$$\mathbf{u}(\mathbf{z}_t, r, t, c) = \frac{1}{t-r} \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau, c) d\tau, \quad (1)$$

where $\mathbf{z}_t$ denotes the latent state at time t , c denotes the text condition and $0 \leq r < t \leq 1$. By definition, the displacement over this interval is $(t-r)\mathbf{u}$, giving $\mathbf{z}_r = \mathbf{z}_t - (t-r)\mathbf{u}(\mathbf{z}_t, r, t, c)$. Taking the full interval from the noise endpoint $t=1$ to the data endpoint $r=0$ yields a one-step latent generator:

$$\hat{\mathbf{z}} = G_\theta(\epsilon, c) = \epsilon - \mathbf{u}_\theta(\epsilon, 0, 1, c), \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (2)$$

The generated latent is converted to a waveform $\hat{\mathbf{a}} = \mathcal{D}(\hat{\mathbf{z}})$, where $\mathcal{D}$ denotes the audio decoding pipeline (VAE [18] and vocoder [19]).

2.2. Distribution Matching with Drifting

Drifting [15] formulates generative learning by evolving the generator-induced distribution q_θ toward the data distribution p_{data} through a distribution-dependent field. For a generated sample $\mathbf{x} \sim q_\theta$, the drifting field is

$$\mathbf{F}_{p_{\text{data}}, q_\theta}(\mathbf{x}) = \frac{\mathbb{E}_{\mathbf{y}^+ \sim p_{\text{data}}} [k(\mathbf{x}, \mathbf{y}^+)(\mathbf{y}^+ - \mathbf{x})]}{\mathbb{E}_{\mathbf{y}^+ \sim p_{\text{data}}} [k(\mathbf{x}, \mathbf{y}^+)]} - \frac{\mathbb{E}_{\mathbf{y}^- \sim q_\theta} [k(\mathbf{x}, \mathbf{y}^-)(\mathbf{y}^- - \mathbf{x})]}{\mathbb{E}_{\mathbf{y}^- \sim q_\theta} [k(\mathbf{x}, \mathbf{y}^-)]}, \quad (3)$$

where $\mathbf{y}^+$ and $\mathbf{y}^-$ denote positive (real) and negative (generated) reference samples drawn from p_{data} and q_θ , respectively, and $k(\cdot, \cdot)$ denotes the similarity kernel. For conditional generation, the construction can be applied to $p_{\text{data}}(\cdot | c)$ and $q_\theta(\cdot | c)$. In practice, the expectations are approximated empirically using finite samples, and the drifting field may be evaluated in a feature space.

3. DRIFTAUDIO

DriftAudio post-trains a text-to-audio generator by applying Drifting [15] to its marginal audio distribution in a frozen feature space. As illustrated in Fig. 1, each generated sample remains conditioned

on its text input, while real and generated samples collected across text conditions are used to estimate the marginal drifting field. A frozen audio encoder defines the feature space in which the field is constructed, and only the generator parameters are updated.

3.1. Marginal Drifting under Free-Form Conditioning

For text-conditioned TTA, applying Drifting at the condition-specific level requires estimating $p_{\text{data}}(\cdot | c)$. Since $p_{\text{data}}(\cdot | c)$ cannot be reliably estimated from one or a few samples per free-form text condition, we instead apply Drifting to the marginal distributions. Given text conditions $\{c_n\}_{n=1}^N$, the marginal data and generator-induced distributions are approximated by

$$p_{\text{data}}(\cdot) \approx \frac{1}{N} \sum_{n=1}^N p_{\text{data}}(\cdot | c_n), \quad q_\theta(\cdot) \approx \frac{1}{N} \sum_{n=1}^N q_\theta(\cdot | c_n). \quad (4)$$

Hereafter, p_{data} and q_θ denote these marginal audio distributions.

We approximate these marginal distributions with finite real and generated samples collected across text conditions, and estimate the corresponding drifting field in Eq. (3) in a frozen feature space defined by an audio encoder ϕ . In this feature space, the drifting field is constructed from three sets: positive samples, negative samples, and the current generated samples. We define the positive and negative sets as

$$\mathcal{Y}^+ = \{\mathbf{y}_i^+ = \phi(\mathbf{a}_i)\}_{i=1}^{N_+}, \quad \mathcal{Y}^- = \{\mathbf{y}_i^- = \phi(\hat{\mathbf{a}}_i)\}_{i=1}^{N_-}. \quad (5)$$

At each optimization step, $\mathcal{Y}^+$ is resampled without replacement from the full set of real training features, while $\mathcal{Y}^-$ is maintained as a FIFO bank initialized with pre-generated features and refreshed using the current generated samples, which are defined as

$$\mathcal{X} = \{\mathbf{x}_i = \phi(\hat{\mathbf{a}}_i)\}_{i=1}^Q. \quad (6)$$

All feature samples lie in $\mathbb{R}^D$, where D denotes the feature dimension. Here, $\mathcal{Y}^-$ is maintained as a bank of detached features from recently generated samples, avoiding regeneration of a large set at every step, whereas $\mathcal{X}$ contains the current generated samples on

which the field is evaluated. The sets $\mathcal{X}$, $\mathcal{Y}^+$, and $\mathcal{Y}^-$ are then used to construct the marginal drifting field.

Following Drifting [15], let w_{ij}^+ and w_{ij}^- denote the kernel-derived weights between $\mathbf{x}_i$ and the samples in $\mathcal{Y}^+$ and $\mathcal{Y}^-$, respectively, with $S_i^+ = \sum_j w_{ij}^+$ and $S_i^- = \sum_j w_{ij}^-$. The corresponding denominator-cleared marginal drifting field is

$$\mathbf{F}_i = S_i^- \sum_j w_{ij}^+ (\mathbf{y}_j^+ - \mathbf{x}_i) - S_i^+ \sum_j w_{ij}^- (\mathbf{y}_j^- - \mathbf{x}_i). \quad (7)$$

3.2. Drifting-Based Post-Training

The previous subsection defines the marginal drifting field $\mathbf{F}_i$ in feature space. We convert this field into a generator training objective by defining a detached target for each generated feature,

$$\mathbf{t}_i = \text{sg}(\mathbf{x}_i + \mathbf{F}_i), \quad (8)$$

where $\text{sg}(\cdot)$ denotes stop-gradient.

To account for the numerical scale of the feature space, we normalize the regression by the average distance between the current generated samples and the reference samples,

$$s = \frac{1}{Q(N_+ + N_-)} \sum_{i=1}^Q \sum_{j=1}^{N_+ + N_-} \|\mathbf{x}_i - \mathbf{y}_j\|_2, \quad (9)$$

where $\mathbf{y}_j \in \mathcal{Y}^+ \cup \mathcal{Y}^-$ and $\|\cdot\|_2$ denotes the Euclidean norm. The post-training objective is then

$$\mathcal{L}_{\text{Drift}} = \frac{1}{Q} \sum_{i=1}^Q \left\| \frac{\mathbf{x}_i - \mathbf{t}_i}{\text{sg}(s)} \right\|_2^2. \quad (10)$$

The target and normalization factor are detached during differentiation, while gradients from $\mathbf{x}_i$ are backpropagated via the frozen audio encoder and decoding pipeline to update only the generator parameters.

4. EXPERIMENTS

4.1. Experimental Setup

Dataset: We conducted all experiments on AudioCaps [20]. Because some source audio was unavailable, our locally recovered subset contained 45,173 training clips (~ 125.5 hours), 445 validation clips, and 877 test clips. The validation set was used for hyperparameter tuning and checkpoint selection, while the test set was evaluated only after the configuration was fixed. All clips are processed as 10-s segments following MeanAudio [11].

Implementation details: We applied the proposed DriftAudio framework to the MeanAudio-S-Full [11] (denoted MeanAudio below) and FdAudio [13] checkpoints. We used a frozen PANNs Cnn14 encoder [21] at 16kHz to extract 2048-dimensional audio features. We precomputed PANNs features for all 45,173 real training samples, from which $N_+ = 2048$ positive references are sampled without replacement at each optimization step. We maintained $N_- = 1024$ detached generated references in a FIFO bank initialized from the corresponding pretrained generator. Each step uses $Q = 64$ current-generated samples, whose detached pre-update features are used to refresh the FIFO bank. We use AdamW [22] with a learning rate of 10^{-6} , a weight decay of 10^{-6} , and gradient clipping at 1.0. Checkpoint selection was based on validation FAD, yielding checkpoints after 4,400 and 1,800 optimization steps when

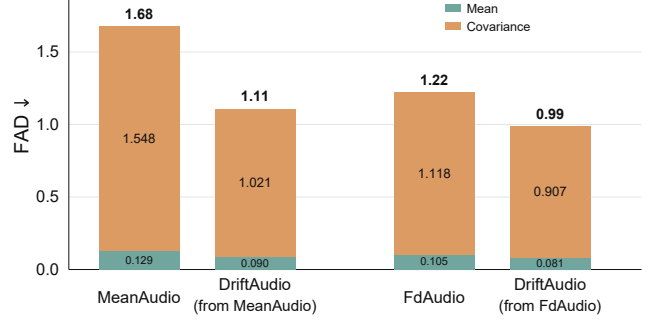


Fig. 2. Mean and covariance components of FAD for the four evaluated models. Numbers above the bars denote total FAD.

starting from MeanAudio and FdAudio, respectively. For memory efficiency, we computed the detached field without retaining the generator computation graph and exactly replayed the same text–noise pairs during backpropagation.

Metrics: Following MeanAudio [11], we reported FAD in the VGGish feature space [23, 24], FD in the 2048-dimensional PANNs feature space [21], KL and IS from PANNs outputs [25], and CLAP as the mean text–audio cosine similarity from LAION-CLAP [26]. The lower the FAD, FD, and KL values, the better, while the higher IS and CLAP values, the better. We additionally reported latency for generating a batch of 8 audio clips [27] on an NVIDIA L40 GPU.

4.2. Main Results and Comparisons

We compared DriftAudio with representative one-step TTA systems and an existing distributional post-training method. All comparison models were evaluated from their publicly released checkpoints, including ConsistencyTTA [6], SoundCTM [8], MeanAudio [11], and FdAudio [13]. In particular, FdAudio is obtained by further post-training MeanAudio on an unreleased 80,000-sample subset randomly drawn from the AudioCaps–WavCaps pool [13, 28]. All baseline models were reevaluated on the same locally recovered test set for consistency, and may therefore differ modestly from those reported in the original publications.

Table 1 summarizes the results. The upper block compares representative one-step TTA systems, while the lower block focuses on distributional post-training. All results are reported with NFE = 1. Since FdAudio and DriftAudio modify only the model weights during post-training, they preserve the inference procedure of MeanAudio and introduce no additional inference-time computation.

DriftAudio (from MeanAudio) and DriftAudio (from FdAudio) denote post-training starting from the corresponding checkpoints. Overall, DriftAudio consistently improves FAD, FD, and KL from both starting checkpoints. Starting from MeanAudio, it reduces FAD from 1.68 to 1.11 and FD from 15.43 to 12.71, while achieving the lowest KL of 1.21. Starting from FdAudio, it further reduces FAD from 1.22 to 0.99 and FD from 13.73 to 12.61. These gains are accompanied by trade-offs in IS and CLAP: when starting from MeanAudio, CLAP improves slightly while IS decreases; when starting from FdAudio, both IS and CLAP decrease. These results demonstrate the effectiveness of DriftAudio as a distributional post-training method for one-step TTA generation.

DriftAudio performs in the PANNs feature space, whereas FAD is computed using VGGish features. We further analyze the mean

Table 1. Results of one-step TTA generation on AudioCaps. The upper block compares representative one-step TTA systems, while the lower block compares distributional post-training methods.

Method	NFE	FAD ↓	FD ↓	KL ↓	IS ↑	CLAP ↑	Latency (s) ↓
<i>General one-step TTA systems</i>							
ConsistencyTTA [6]	1	2.25	22.58	1.39	8.91	0.287	0.875
SoundCTM [8]	1	1.94	18.68	1.31	8.58	0.275	0.964
MeanAudio [11]	1	1.68	15.43	1.27	10.36	0.322	1.271
<i>Distributional post-training</i>							
DriftAudio (from MeanAudio)	1	1.11	12.71	1.21	9.83	0.325	1.271
FdAudio [13]	1	1.22	13.73	1.27	10.84	0.352	1.271
DriftAudio (from FdAudio)	1	0.99	12.61	1.24	10.17	0.335	1.271

Table 2. Text–audio alignment on generated samples.

Method	CLAP ↑	R@1 ↑	R@5 ↑
MeanAudio	0.322	32.5	71.0
DriftAudio (from MeanAudio)	0.325	34.5	74.2
FdAudio	0.352	38.7	74.9
DriftAudio (from FdAudio)	0.335	36.7	73.1

and covariance components of FAD to examine whether the distributional improvement extends beyond the feature space used for post-training. Fig. 2 shows that DriftAudio reduces both components from both starting checkpoints. Starting from MeanAudio, the covariance decreases substantially from 1.548 to 1.021, together with a reduction in the mean from 0.129 to 0.090. A similar trend is observed when starting from FdAudio. These results show that the distributional improvement is also observed in the VGGish feature space, which is not used by the DriftAudio objective.

4.3. Analysis and Ablation Study

Text–audio alignment. Since DriftAudio estimates a marginal drifting field across text conditions, we examined whether marginal post-training preserves the dependence of generated audio on its conditioning text. Following audio–text retrieval evaluation [29], we treated each generated audio sample as a query and ranked candidate captions by cosine similarity in the CLAP embedding space. Duplicate captions in the test set were merged, resulting in 866 unique candidate captions. We reported audio-to-text Recall@1 (R@1) and Recall@5 (R@5), indicating whether the matched caption appears among the top 1 or top 5 retrieved candidates

Table 2 shows that FdAudio achieves the highest scores across all three metrics, including an R@1 of 38.7%. Starting from MeanAudio, DriftAudio improves both CLAP and retrieval performance, increasing R@1 from 32.5% to 34.5%. Starting from FdAudio, DriftAudio shows a modest decrease relative to FdAudio but remains above MeanAudio on all three metrics. Overall, the retrieval results suggest that marginal distributional post-training does not substantially weaken text–audio alignment.

Ablation study. We evaluated a continued-training baseline and three DriftAudio design choices, all starting from the released FdAudio checkpoint and using the same validation-based checkpoint selection protocol. The **default configuration** used per-step real-reference resampling, PANNs features, $N_+ = 2048$ real references, and $N_- = 1024$ generated references. We compared it with: (1) **Continued FdAudio training**, which continued post-training using the original FdAudio objective; (2) **Fixed real references**, which used the same $N_+ = 2048$ real references throughout train-

Table 3. Ablation of key DriftAudio design choices and continued-training baseline from FdAudio.

Variant	FAD ↓	FD ↓	CLAP ↑
Continued FdAudio training	1.40	13.57	0.358
Fixed real references	1.04	12.94	0.343
AudioMAE features	1.02	13.41	0.333
Larger reference sets	0.98	12.69	0.338
Default configuration	0.99	12.61	0.335

ing; (3) **AudioMAE features**, which replaced PANNs with AudioMAE [30]; and (4) **Larger reference sets**, which doubled the reference sizes to $N_+ = 4096$ and $N_- = 2048$. For each variant, we selected the checkpoint with the lowest validation FAD.

Table 3 shows that the default configuration provides a practical and effective setting: real-reference resampling improves FAD and FD, the drifting formulation remains effective in a different audio feature space, and increasing the reference-set size brings only marginal gains. Continued FdAudio training slightly improves FD and CLAP but worsens FAD from 1.22 to 1.40, whereas DriftAudio reduces it to 0.99. This control shows that the gains cannot be explained by additional optimization alone, supporting the marginal Drifting objective. Using fixed real references increases FAD from 0.99 to 1.04. Replacing PANNs with AudioMAE still yields competitive performance, suggesting that the formulation can operate across different audio feature spaces, although PANNs performs better in the current setting. Doubling the reference-set sizes yields a comparable FAD of 0.98, but increases the median training time per optimizer step from 23.7 s to 49.5 s on a single NVIDIA L40 GPU (2.09×). The default reference sizes provide a practical balance between distributional performance and training cost.

5. CONCLUSION

We introduced DriftAudio, a distributional post-training method for pretrained one-step TTA generators that applies Drifting to the marginal audio-feature distribution. DriftAudio retains text-conditioned generation while estimating the drifting field from real and generated samples collected across text conditions. On AudioCaps, DriftAudio consistently improves FAD, FD, and KL when starting from both MeanAudio and FdAudio, without substantially weakening text–audio alignment. These results suggest that marginal Drifting may provide a general distributional post-training principle for conditional generative models. Future work will extend the current single-encoder formulation to multiple complementary audio representations and include subjective perceptual evaluation of the generated audio.

6. REFERENCES

- [1] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi, “AudioGen: Textually guided audio generation,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [2] Haohe Liu, Zehua Chen, Yi Yuan, Xinhao Mei, Xubo Liu, Danilo Mandic, Wenwu Wang, and Mark D. Plumbley, “AudioLDM: Text-to-audio generation with latent diffusion models,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, vol. 202, pp. 21450–21474.
- [3] Haohe Liu, Yi Yuan, Xubo Liu, Xinhao Mei, Qiuqiang Kong, Qiao Tian, Yuping Wang, Wenwu Wang, Yuxuan Wang, and Mark D. Plumbley, “AudioLDM 2: Learning holistic audio generation with self-supervised pretraining,” *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 32, pp. 2871–2883, 2024.
- [4] Chia-Yu Hung, Navonil Majumder, Zhifeng Kong, Ambuj Mehrish, Amir Zadeh, Chuan Li, Rafael Valle, Bryan Catanzaro, and Soujanya Poria, “TangoFlux: Super fast and faithful text to audio generation with flow matching and CLAP-ranked preference optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [5] Wenhao Guan, Kaidi Wang, Wangjin Zhou, Yang Wang, Feng Deng, Hui Wang, Lin Li, Qingyang Hong, and Yong Qin, “LAFMA: A latent flow matching model for text-to-audio generation,” in *Interspeech*, 2024, pp. 4813–4817.
- [6] Yatong Bai, Trung Dang, Dung N. Tran, Kazuhito Koishida, and Somayeh Sojoudi, “ConsistencyTTA: Accelerating diffusion-based text-to-audio generation with consistency distillation,” in *Interspeech*, 2024, pp. 3285–3289.
- [7] Huadai Liu, Rongjie Huang, Yang Liu, Hengyuan Cao, Jialei Wang, Xize Cheng, Siqi Zheng, and Zhou Zhao, “AudioLCM: Efficient and high-quality text-to-audio generation with minimal inference steps,” in *Proc. ACM Int. Conf. Multimedia (MM)*, 2024, pp. 7008–7017.
- [8] Koichi Saito, Dongjun Kim, Takashi Shibuya, Chieh-Hsin Lai, Zhi Zhong, Yuhta Takida, and Yuki Mitsufuji, “SoundCTM: Unifying score-based and consistency models for full-band text-to-sound generation,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.
- [9] Junqi Zhao, Jinzheng Zhao, Haohe Liu, Yun Chen, Lu Han, Xubo Liu, Mark D. Plumbley, and Wenwu Wang, “AudioTurbo: Fast text-to-audio generation with rectified diffusion,” in *Interspeech*, 2025.
- [10] Huadai Liu, Jialei Wang, Rongjie Huang, Yang Liu, Heng Lu, Zhou Zhao, and Wei Xue, “FlashAudio: Rectified flow for fast and high-fidelity text-to-audio generation,” in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Vol. 1: Long Papers, 2025, pp. 13694–13710.
- [11] Xiquan Li, Junxi Liu, Yuzhe Liang, Zhikang Niu, Wenxi Chen, and Xie Chen, “MeanAudio: Fast and faithful text-to-audio generation with mean flows,” in *Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Vol. 1: Long Papers, 2026, pp. 14378–14393.
- [12] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He, “Mean flows for one-step generative modeling,” in *Adv. Neural Inf. Process. Syst.*, 2025, vol. 38.
- [13] Kuan-Po Huang, Bo-Ru Lu, Ho-Lam Chung, Shih-Hsin Wang, and Hung-Yi Lee, “FdAudio: Meanflow-anchored fr chet-distance post-training for one-step text-to-audio generation,” *arXiv preprint arXiv:2607.10421*, 2026.
- [14] Jiawei Yang, Zhengyang Geng, Xuan Ju, Yonglong Tian, and Yue Wang, “Representation fr chet loss for visual generation,” *arXiv preprint arXiv:2604.28190*, 2026.
- [15] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He, “Generative modeling via drifting,” *arXiv preprint arXiv:2602.04770*, 2026.
- [16] Liang Xu, Diego Caviedes-Nozal, W. Bastiaan Kleijn, Longfei Felix Yan, and Rasmus Kongsgaard Olsson, “Speech enhancement based on drifting models,” in *Interspeech*, 2026, pp. 1369–1374.
- [17] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le, “Flow matching for generative modeling,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [18] Jiawei Huang, Yi Ren, Rongjie Huang, Dongchao Yang, Zhenhui Ye, Chen Zhang, Jinglin Liu, Xiang Yin, Zejun Ma, and Zhou Zhao, “Make-An-Audio 2: Temporal-enhanced text-to-audio generation,” *arXiv preprint arXiv:2305.18474*, 2023.
- [19] Sang-Gil Lee, Wei Ping, Boris Ginsburg, Bryan Catanzaro, and Sungroh Yoon, “BigVGAN: A universal neural vocoder with large-scale training,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [20] Chris Dongjoo Kim, Byeongchang Kim, Hyunmin Lee, and Gunhee Kim, “AudioCaps: Generating captions for audios in the wild,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Language Technol. (NAACL-HLT)*, 2019, pp. 119–132.
- [21] Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley, “PANNs: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 28, pp. 2880–2894, 2020.
- [22] Ilya Loshchilov and Frank Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [23] Kevin Kilgour, Mauricio Zuluaga, Dominik Roblek, and Matthew Sharifi, “Fr chet audio distance: A reference-free metric for evaluating music enhancement algorithms,” in *Interspeech*, 2019, pp. 2350–2354.
- [24] Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, R. Channing Moore, Manoj Plakal, Devin Platt, Rif A. Saurous, Bryan Seybold, Malcolm Slaney, Ron J. Weiss, and Kevin W. Wilson, “CNN architectures for large-scale audio classification,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2017, pp. 131–135.
- [25] Tim Salimans, Ian J. Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen, “Improved techniques for training GANs,” in *Adv. Neural Inf. Process. Syst.*, 2016, vol. 29, pp. 2226–2234.
- [26] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Marianna Nezhurina, Taylor Berg-Kirkpatrick, and Shlomo Dubnov, “Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [27] Kuan-Po Huang, Shu-Wen Yang, Huy Phan, Bo-Ru Lu, Byeonggeun Kim, Sashank Macha, Qingming Tang, Shalini Ghosh, Hung-Yi Lee, Chieh-Chi Kao, and Chao Wang, “IMPACT: Iterative mask-based parallel decoding for text-to-audio generation with diffusion modeling,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, vol. 267, pp. 26002–26019.
- [28] Xinhao Mei, Chutong Meng, Haohe Liu, Qiuqiang Kong, Tom Ko, Chengqi Zhao, Mark D. Plumbley, Yuxian Zou, and Wenwu Wang, “WavCaps: A ChatGPT-assisted weakly-labelled audio captioning dataset for audio-language multimodal research,” *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 32, pp. 3339–3354, 2024.
- [29] Andreea-Maria Oncescu, A. Sophia Koepke, Jo o F. Henriques, Zeynep Akata, and Samuel Albanie, “Audio retrieval with natural language queries,” in *Proc. Interspeech*, 2021, pp. 2411–2415.
- [30] Po-Yao Huang, Hu Xu, Juncheng Li, Alexei Baevski, Michael Auli, Wojciech Galuba, Florian Metze, and Christoph Feichtenhofer, “Masked autoencoders that listen,” in *Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 28708–28720.